

Serializability: ensures that the schedule for the concurrent execution of several transactions should yield consistent results

NoSQL DBs give you BASE instead: Basic Availability, Soft state, Eventual consistency. Weaker guarantees, but faster and more scalable.

ACID	BASE
Strong consistency	Weak consistency – stale data OK
Isolation	Availability first
Focus on “commit”	Best effort
Nested transactions	Approximate answers OK
Availability?	Aggressive (optimistic)
Conservative (pessimistic)	Simpler!
Difficult evolution (e.g. schema)	Faster
	Easier evolution

Transaction Log: simple text file. Every change is written here before it's applied to the table (Write-Ahead Log / WAL). If something crashes, the log lets you redo committed changes or rollback uncommitted ones. "A log text file saves us."

Concurrency Problems (what happens without coordination)

Dirty read/Uncommitted data - reading another transaction's uncommitted changes

Lost update - two transactions read the same value, both compute new values, both write back - one clobbers the other

Unrepeatable read/improper retrieval - you read a value, another transaction changes it, you read again and get something different

Scheduler-establishes order in which operations are executed within concurrent transactions. Creates serialisation schedule. Ensures serializability and isolation of transactions

Locking: a lock manager - a gatekeeper process inside the DB. ensures isolation. Guarantees exclusive use of data item to current transaction

Two types of locks:Shared lock (read lock): Many transactions can hold simultaneously - they're all just reading

Exclusive lock (write lock): Only ONE transaction gets it. All readers blocked until the write finishes

Binary lock: has 2 states locked(1) and unlocked(0). If an obj is locked by a transaction, no other transaction can use that object. If an object is unlocked, any transaction can lock object for its use.

Levels of locking: db, table, page/diskpage, row, field

Optimistic concurrency control:Don't lock at all. Assume conflicts are rare. Check the log after the fact to see if anything went wrong. Works for rarely-modified data (e.g., a database of ancient mummies - "almost nobody else is modifying that").

Without an index, finding a row means scanning the entire table - O(n). With an index, much faster. "Avoid full table scans like the plague." Example of low sparsity: STU_SEX gender column in a table is either M or F, so indexing this is useless (we'd only have 2

keys, each with 1000s of records!). On the other hand, DOB is a column with relatively higher sparsity, and is worth creating an index

Index selectivity: a measure of how likely an index will be used in a query. So we strive to create indexes that have high selectivity

Indexes are useful when: an indexable column occurs in a WHERE or HAVING search expression; an indexable column appears in a GROUP BY or ORDER BY clause; MAX or MIN is applied to an indexable column; there is high data sparsity on an indexable column. Worth creating indexes on single columns that appear in WHERE, HAVING, ORDER BY, GROUP BY and join conditions.

Why does an expression like PRICE*1.2 > 50 make the index on PRICE unusable? A: Indexes store raw column values, not computed expressions. The optimizer can't map PRICE*1.2 back to a position in the PRICE index - it would have to scan the whole table and compute the expression for every row. Always index the raw column; move arithmetic to the other side of the comparison instead (PRICE > 41.67).

What is a two-level index? Give an example. A: A two-level index chains two separate indexes. First index filters on column A (e.g., state), second index filters within those results on column B (e.g., salary). For example: find all California employees earning over \$300K - first use the state index to get California rows, then use the salary index within those. Which is "primary" depends on which column your queries filter first most often.

Three Types of Indexes

B-tree index: Best for columns with high sparsity (many distinct values) - names, GPA, salary, dates. Builds a sorted binary tree. Searching = start at root, go left (smaller) or right (larger) → O(log n). example: searching for "Server" in a name column - compare to middle value, go right half, compare again, find it in ~3 steps vs scanning all rows

Hash index: A hash function maps a column value directly to a memory slot → O(1) - constant time regardless of table size

Best for exact equality lookups (WHERE name = 'Alice'). NOT useful for range queries. "Without even sorting, the function tells you exactly where the data lives"

Bitmap index: Best for columns with low sparsity (few distinct values) - region (NE/NW/SE/SW), gender, boolean flags. Creates a bit vector for each possible value (one bit per row). Hardware AND/OR operations find matches blazing fast. Prof's example: street direction column with only 4 compass values - store as 4-bit pattern per row, hardware AND to find all "NW" customers instantly.

in the bitmap index, the number of rows will equal the # of rows in the actual table, and the number of columns, and the column names, will equal the possible values for the table column we are indexing

Column Sparsity Rule: High sparsity (many distinct values like GPA with ~401 possible values, or birthday with 365 values) → use B-tree index. Low sparsity (few distinct values like gender with 2 values, region with 4) → use bitmap index. Prof's point: social media companies index birthday columns - at midnight they intersect "born today" with your friends list to send birthday notifications

SQL-Level Optimization Tips

Use simple columns or literals as operands

Equality comparisons (=) are faster than inequality (<, >, BETWEEN)

In AND conditions, put the condition most likely to be FALSE first (short-circuit)

In OR conditions, put the condition most likely to be TRUE first

Avoid NOT - optimizer can't use an index on a negation

OLTP (Online Transaction Processing): fast, row-level inserts/updates. Normalized. Day-to-day operations

OLAP (Online Analytical Processing): complex aggregations over historical data. Denormalized for speed. This is BI.

Data Warehouse: A separate database loaded with historical transactional data, optimized for analysis. The "microscope" of the business - lets you zoom in and out on data.

Star Schema - The Central Design Pattern

A fact table in the middle, surrounded by dimension tables.

Fact table: Contains actual measurements/events (each row = one transaction/sale). Highly denormalized. Columns are mostly foreign keys pointing to dimension tables plus measures (quantity, price, total). Always stays denormalized. "3,000,000 records" - huge.

Dimension tables: Provide the "who/what/when/where" context. Customer, Product, Time, Location. Much smaller tables. Many-to-one relationship with the fact table.

Prof's Starbucks example: the sales fact table has TIME_ID, LOC_ID, CUST_ID, PROD_ID, SALES_TOTAL. The store ID is Starbucks #1 (Pike Place, Seattle). To find out who the customer is - go to the Customer dimension table. To find the product name - go to Product dimension. The dimension tables "qualify" (give meaning to) the raw IDs in the fact table.

Snowflake Schema

Dimension tables are normalized into chains. Location dimension becomes City → State → Region (three linked tables). Cleaner design, but requires traversing the chain to roll up. Enables hierarchical drill-down/roll-up.

Drill-down and Roll-up (the "conceptual microscope")

Made possible by dimension hierarchies.

Roll-up: Zoom out - City → State → Region → Country (aggregate more data, less detail)

Drill-down: Zoom in - Year → Quarter → Month → Day (more detail)

Example: Looking at annual revenue? Roll up from daily transactions. Want to know which Q3 month underperformed? Drill down from quarterly to monthly.

four different ways in which we can organize (structure) a data warehouse

1. star schema: fact table FKs point to a single level of dimension tables (points of a star)
2. snowflake schema: each dimension table can be normalized to create a 1:M chain
3. the fact table can be supplanted with all the columns in the star/snowflake dimensions
4. a separate fact table can be created for each attribute in a dimension hierarchy

To denormalize a fact table (#3 above), we simply add extra 'dimension' columns to it, and fill them with redundant data - this permits fast queries (no joins needed) at the expense of disk space (and cleanliness of design). Instead of having a fact table pointing to denormalized star dimensions (#1), or a fact table with lowest attrs pointing to a chain of rolled-up attrs, ie. snowflake schema (#2), or creating a denormalized fact table (#3), we can create multiple fact tables, one for each level in an attr hierarchy (#4) - it is a different form of denormalization, where the redundant data is stored in physically separate tables.

TWO BI-oriented SQL extensions

ROLLUP and CUBE are GROUP BY modifiers - they help generate subtotals for a list of specified columns. Depending on granularity of the columns (eg. US_REGION vs STORE_NUMBER), these subtotals help provide a rolled-up (aggregated) or drilled-down (detailed) analysis of data. ROLLUP used to generate aggregates by diff dimensions. Enables subtotal for each col listed except for the last one which gets a grand total. Order of col list imp. CUBE used to generate aggregates by the listed columns. Includes the last column.

In BI, why do we need the dimension tables surrounding the fact table (i.e., the star schema)?A: The fact table contains raw measurements/events (quantities, prices, IDs) but the IDs alone have no descriptive meaning. Dimension tables "qualify" the facts - they provide the who/what/when/where context. For example, a SALES fact table has CUST_ID, but you need the CUSTOMER dimension to know the customer's name, region, age segment. Without dimensions, you can't answer "what" the numbers mean.

How has BI evolved? Discuss two ways.A: (1) Evolution of outcomes: from simple reporting and cost analysis → predictive analytics and AI-driven business decisions using ML models. (2) Evolution of accessibility/delivery: from static desktop reports printed periodically → cloud-hosted, mobile-accessible, real-time dashboards available anywhere on any device. Also: from batch

ETL (nightly/weekly) → near-real-time streaming pipelines (Kafka/Storm).

Traditional BI is highly inadequate today - discuss 5 aspects.

1. Data Types Traditional BI (star/snowflake schema, ETL) handles only structured tabular data. Cannot handle unstructured data (images, video, audio, PDFs). Cannot handle geospatial data natively. Modern data is overwhelmingly unstructured.
2. Data Collection Traditional ETL is batch processing - collect data, transform it, load it periodically (nightly, weekly). Cannot handle real-time streaming data from IoT sensors, social media feeds, clickstreams. Events that need immediate response (fraud detection, stock trading) cannot wait for the next batch.
3. Storage Star/snowflake schema has rigid, fixed structure - schema changes are painful (ALTER TABLE on billions of rows). Cannot scale horizontally easily. Modern data lakes / NoSQL / cloud storage are needed.
4. Processing Traditional BI produces results after a delay - not real-time. Cannot integrate machine learning models for predictive analytics. Modern: cloud + GPU processing, real-time stream processing (Kafka/Storm), ML integrated into the pipeline.
5. Results Consumption Traditional BI output: static reports and dashboards, desktop-only, periodic snapshots. Modern: dynamic visualizations, mobile dashboards (consume insights anywhere), alerts/notifications when conditions are met, NL query interfaces (ask questions in plain English via agentic AI/MCP), continuous real-time updates.

Spatial Data? Any data with a location - a point, line, or polygon. Geospatial = specifically uses Earth coordinates (lat/long). Not all spatial is geospatial: DNA mapping, galaxy clusters, organs in the body are also spatial but not geospatial.

Examples of geospatial data: spread of disease, risk of disease, drug overdoses, census data, income distribution, home prices, starbucks locations, real-time traffic, crime data, agri land use

examples of how three city government departments would use spatial data. A: Energy: locations of power plants, transmission lines, energy storage - for maintenance routing and future infrastructure planning. Water: pipeline network maps, flood risk zones, watershed catchment areas - for maintenance and disaster planning. Public Safety: camera placement, emergency room/hospital locations, crime hotspot maps, patrol routing optimization.

Spatial data creations: CAD: user creation, reverse engineering; maps: cartography(survey,plot), satellite imagery, copter/drone imagery, driving around, walking around

Spatial data description: points / vertices / nodes; polyline / arcs / linestrings; polygons / regions; pixels / raster

The Three Spatial Data Types

Point - a single (x, y) location (a restaurant, a crime scene, a taxi pickup)

Line - a path with length (a road, a river, a flight path)

Polygon - a closed area (a building footprint, a city boundary, a ZIP code)

Spatial relationships: topology-based [using defns of boundary, interior, exterior]; metric-based [distance/Euclidian, angle measures]

Direction-based; network-based [eg. shortest path]

Topological relationships could be further grouped like so:proximity, overlap, containment

In 1D (and higher), spatial relationships can be expressed using 'intersects', 'crosses', 'within', 'touches' (these are T/F predicates).

What do spatial databases and vector databases have in common, architecturally?A: Both use multi-dimensional spaces to represent data (2D for spatial, high-dimensional for vectors). Both require specialized hierarchical indexing to avoid brute-force search. Spatial DBs use R-trees (MBRs nested in a hierarchy); vector DBs use HNSW (Hierarchical Navigable Small World - layers of graphs at decreasing granularity). Core idea is the same: prune the search space hierarchically so you never scan everything.

Why does spatial data need a two-level indexing scheme?

A: Level 1 = MBR/R-tree filters candidate geometries (fast, approximate). Because sorting isn't naturally defined on 2D shapes, you first simplify them to bounding rectangles you CAN sort/compare. Level 2 = exact geometry check on the survivors from level 1 (precise, slower). Level 1 prunes huge portions of the search space; level 2 confirms actual spatial matches.

Application where spatial + GenAI combine: NeRFs (Neural Radiance Fields) - synthesizing new views of a 3D scene from photos. Google's Photorealistic 3D Tiles (spatial data) + GenAI to render seamless drone-like flythrough paths. Urban planning / city simulation using GenAI to generate realistic environments from spatial data.

Set functions on Spatial Data:

Return a geometry: *intersect*; *buffer*; *centerPoint*; *convexHull*;

Union of two geometries: Merge two regions into one. Two countries uniting → the shared border disappears, a new larger polygon forms; *Intersection* of two geometries: The overlapping area only. Prof's example: the DMZ between North Korea and South Korea - both countries claim it, so it's the intersection of their claimed territories. India-China actual line of control - both countries' maps show that region as theirs; *Difference* (A – B): Everything in A that is NOT in B. The yellow region minus the blue overlap. Not symmetric; *XOR* (symmetric difference): Union minus intersection - the parts of each polygon that don't overlap with the other.

Spatial Metric Operations (Non-Boolean) These return numbers, so they break closure.

Distance: How far between two points? (Breaks closure - returns a number)

Area: How large is a polygon?

Perimeter / Length: How long is a polygon's boundary or a line?

Buffer: Creates a new polygon by expanding a geometry by a specified distance (e.g., a 500m buffer around a school)

Closure in spatial algebra: Most spatial operations preserve the geometry type - geometry in, geometry out. But distance calculations break closure (geometry in, number out). "If I ask how long is the 71 freeway, I get one number, 2000 miles - closure breaks."

Spatial operators: These answer yes/no questions about two geometries. Implemented as functional extensions in SQL

Topological operators- Inside; Covers; Covered by; Overlap boundary; **Touch:** Do two polygons share a boundary but not interior? (e.g., two neighboring states); **Contains** / Within: Is one polygon entirely inside another? (Vatican City is within Rome); **Equals:** Are two geometries identical? (Same polygon, just with vertices listed in different order); **Disjoint:** Do two geometries share nothing? (Is Palo Alto adjacent to Los Angeles? No - disjoint)

Distance operators: within distance; nearest neighbor

SQL with spatial extensions looks like: WHERE SDO_TOUCH(a.geom, b.geom) = 'TRUE'

Spatial Queries (Computational Geometry)

Containment: Is point P inside polygon A?

Proximity / nearest neighbor: Find the 5 closest hospitals to me

Intersection: Do two polygons overlap?

Range: Find all restaurants within 2 miles (bounding box query)

Can B Trees index spatial data? In short, YES, if we pair it up with a 'z curve' indexing scheme (using a space-filling curve)

Spatial Indexing - R-trees (The Important One)

Normal 1D indexes can't handle 2D data. R-trees solve this using Minimum Bounding Rectangles (MBRs) - the smallest rectangle that fully contains a spatial object. For 3d it is MBV. MBRs are nested into a hierarchy:

Top level: one big MBR covering many objects

Next level: several smaller MBRs covering sub-regions

Leaf level: MBRs tightly wrapping individual objects

Query: Does my query region overlap this top-level MBR? No → skip the entire subtree. Yes → recurse. Huge portions of the data get pruned without looking at them. Variations: R+ tree, R* tree, Buddy trees, Packed R trees

Quadtree: recursively subdivides 2D space into 4 quadrants - but only where data/detail exists; sparse regions remain large undivided cells. Octree = the 3D equivalent (8 subdivisions), used for volumetric data like medical scans (CT/MRI) or 3D models - only subdivides where surface/data detail exists (e.g., a 3D scan of a brain or the Stanford Bunny model).

Z-curve indexing: maps 2D (x,y) points to a 1D ordered value by interleaving their binary coordinates, allowing a standard B-tree to index spatial data without needing a dedicated R-tree. Mostly of academic interest; not common in industry (Apple's FoundationDB is a notable exception).

What is the main limitation of MBRs? What is the better alternative? R-tree / MBR limitation: MBRs waste space. The bounding rectangle is always axis-aligned, so for diagonal or irregular shapes (a 45-degree line, a crescent shape, an L-shaped building footprint), the MBR includes huge amounts of empty space that isn't part of the actual object. This empty space creates false positives - objects appear to intersect the MBR when they don't actually touch the real geometry. More false positives = more unnecessary detailed calculations = slower queries. **Better alternative:** OBB (Oriented Bounding Box) - a rectangle that can be rotated to align with the actual shape, minimizing wasted space. Or a convex hull - the tightest possible convex polygon around the object. Either wastes much less space than an axis-aligned MBR. (Note: quad-trees, R*-trees, and minimum bounding circles still have similar wasted-space problems - they aren't valid alternatives here.)

Choropleth Maps: A map where every geographic region is shaded/colored by a quantitative variable. Every point on the map has a value.

Classed (discrete): Discrete color buckets (ranges). Hard boundaries between colors. Unclassed (continuous): Smooth color gradient. Ex for unclassified:weather map showing temperature across a country, smoothly fading from blue (cold) to red (hot), with no hard lines between colors.elevation map showing how high the land is, gradually shading from green (low) to brown to white (high mountains).rainfall map showing how wet or dry different areas are, smoothly blending from light to dark.

Examples of good classed choropleth data: drug overdose deaths per 100K by county, unemployment rate by ZIP code, median household income by state, COVID case rate per 100K by country.

Why NoSQL Emerged

Internet-scale data (YouTube, Twitter, Amazon) requires:

Flexible schemas (a tweet and a product listing have nothing in common structurally)

Horizontal scaling (add more servers, not just bigger servers)

Semi-structured/unstructured data (JSON, graphs, documents)

Speed for specific access patterns

no need to perform (expensive) JOIN operations.. Also, queries can be parallelized using MapReduce.

The Four NoSQL Models + Vector

1. Key-Value Store (Redis, DynamoDB, Voldemort) The simplest NoSQL. Only three operations: PUT(key, value), GET(key),

DELETE(key). No deep search API. Values must be simple and small - everything fits in main memory → ultra-fast O(1) access. Use cases: session caches, shopping carts, game leaderboards, Starbucks order queues. Prof: "Like a Python dictionary that lives in memory."

2. Document Store (MongoDB, CouchDB) Conceptually same as k-v, but values can be arbitrarily complex nested JSON/BSON - entire blog posts, arrays of objects, deeply nested structures. Lives on disk (also cached). Rich query API: range queries, geospatial queries, full-text search, deep nested field search. Schema-on-read (not schema-on-write) - different documents can have completely different fields; no upfront table definition. Prof's example: a Medium blog post as one JSON object with keys title, author, body, comments (array of nested objects), tags (array). 100 blog posts from one author = an array of 100 objects. Deep search lets you search for "JavaScript" anywhere - in title, body, comments, author name - all via one API call. Key difference from k-v: document DB intentionally allows disk storage and deep search; k-v intentionally keeps things simple and in-memory.

3. Column-Family Store (Cassandra, HBase) Looks like a table but optimized for column-level analytical operations (sums, averages, aggregations across a column) rather than row-level operations. Great for time-series data and analytics on massive datasets.

4. Graph Database (Neo4j, HyperGraphDB) Data stored as nodes and edges (relationships). No tables. Most schema-free of all - any node can connect to any other with any relationship type. Query language: Cypher (Neo4j proprietary) or TinkerPop (open standard - "like SQL for graphs"). Prof's excitement: "Why didn't Codd think of this? It's revolutionarily simple." An ER diagram with Employee table, Project table, and a bridge table becomes: just Alice node, ProjectA node, wire labeled "works on." Gone is the bridge table, gone are the joins. Used for: social networks, fraud detection (circular transactions form cycles in the graph), knowledge graphs for AI agents, recommendation engines.

5. Vector Database (Pinecone, Weaviate; also MongoDB/Neo4j retrofitted) Stores high-dimensional embedding vectors. Queried by similarity - "find me the 5 vectors most similar to this query vector." Similarity = cosine similarity or Euclidean distance. Indexed using HNSW (Hierarchical Navigable Small World) - navigates layers of graphs at decreasing granularity to quickly find nearest neighbors. Same hierarchical pruning idea as R-trees but in high-dimensional space. Used by all LLM RAG systems.

How would you make a graph database similar to a k-v or document DB? A: Express the graph as JSON. Each node becomes a JSON object: {"id": "Alice", "type": "Person", "name": "Alice"}. Each edge becomes a JSON object: {"from": "Alice", "to": "Bob", "rel": "knows"}. Store these in a document DB and you have effectively represented a graph.

Polyglot persistence means the use of different storage technologies (ie. NoSQL DBs) to store different parts (data) of a single application. These individual data stores are then tied together by the application, using DB APIs or web services APIs. Example: a retail app uses Redis for shopping cart sessions (k-v), MongoDB for product catalog (document), Cassandra for order history analytics (column-family), and Neo4j for recommendation engine (graph).

Why MapReduce Exists Google's search index grew too large for one machine. Solution: fragment the index across thousands of machines, search all simultaneously, combine results. "MapReduce was not invented for databases - it was invented for Google search." "MapReduce is just parallel processing"

The 4 Steps (Map → Combine → Shuffle → Reduce)

1. Map - WHERE THE SPEEDUP HAPPENS Break input data into fragments (horizontal partitions of a table). Run the SAME map function on each fragment simultaneously across the cluster. Each mapper emits (key, value) pairs. Parallelism type: SIMD (Single Instruction, Multiple Data).
2. Combine - OPTIONAL Before sending data over the network, do a local aggregation per mapper. If a mapper emitted (C,1)(C,1)(C,1), combine them to (C,3) before shipping. Reduces network bandwidth. "Combining means local reduction."
3. Shuffle Collects all (key, value) pairs from all mappers. Groups all values for the same key together: (C, [3, 2, 8]). Routes each key's value-list to the correct reducer. This is the "level of indirection" - mappers don't talk to reducers directly, the shuffler coordinates.
4. Reduce Each reducer receives: one key + all its values from across the entire cluster. Aggregates to final result: (C, 13).

What is MapReduce high-level vs low-level coding?

A: Low level: writing map/reduce operations directly, manually configuring mappers and reducers, Java/Python code. High level: using frameworks like Apache Hive (SQL → MapReduce), Apache Pig (dataflow language → MapReduce), Apache Spark (in-memory, faster). The framework constructs the maps/reduces under the hood via a graph compiler.

What is the difference between Kafka and Storm? Kafka: Distributed pub-sub messaging system. Producers write data to "topics." Consumers subscribe to topics. Completely decoupled - producers don't know who the consumers are, consumers don't need to be online when producers write. Brokers store messages with durability guarantees.

Storm: Does stream processing on continuous data (vs MapReduce which processes batches). "Storm does for real-time processing what Hadoop did for batch processing." Kafka = the messaging backbone; Storm = the real-time computation engine on top.

Data mining: Finding patterns in large historical datasets to make predictions or drive decisions. "Should I give this person a loan?" "Is this credit card transaction fraudulent?" "What products should I recommend?"

Use cases:predicting which customers will purchase what products and when; deciding should insurance rates be set to ensure profitability; predicting equipment failures, reducing unnecessary maintenance and increasing uptime to optimize asset performance anticipating resource demands; predicting which customers are likely to leave and what can be done to retain them; detecting fraud minimizing financial risk; increasing response rates for marketing campaigns

What is the connection between BI and data mining? A: DM identifies patterns and relationships in large datasets → BI interprets, visualizes, and acts on them. Together they enable data-informed decisions. DM is the discovery engine; BI is the display and decision layer. Both extract novel, usable information from large volumes of pre-existing data.

Four Types of DM Algorithms:

Classification - labeling data Given past labeled examples, learn to assign a label to new data. "Is this email spam?" "Is this tumor malignant?" "Will this customer churn?" Needs labeled training data (supervised). Algorithms: Decision trees, k-nearest neighbors (kNN), logistic regression, neural networks, naive Bayes.

Clustering - grouping data No labels. Find natural groups in data based on similarity. "I don't know what the groups are - find k clusters for me." Algorithm: k-means - pick k random centroids, assign each point to nearest centroid, recompute centroids, repeat until convergence. Iterative! How to choose k: plot error vs k, look for the "elbow" (point of diminishing returns).

Association - relating data Find items that co-occur frequently. Classic: "people who buy tortilla chips also buy salsa" → put salsa next to chips. Used in: recommendation engines (Amazon "also bought"), store layout optimization. No labeling, no clustering - just co-occurrence patterns.

Regression - coupling/fitting data Fit a mathematical function (line or curve) to the data. Linear regression: fit $y = mx + c$ through the data. Outlier detection: a charge from Finland when you always shop locally = outlier → fraud alert. A neural network IS essentially a very complex nonlinear regression model.

Decision Trees (Classification Algorithm)

Recursively partition data space with if/else splits on features. Each internal node = a feature test. Each leaf = a class label. Very interpretable - you can read why it classified something a certain way. Classification tree (categorical output) vs Regression tree (continuous output) → together = CART algorithms.

Classification vs Clustering

Classification: pre-existing LABELED buckets (you know the categories upfront, model learns to assign new data to them) Clustering: discovers UNLABELED groups (you don't know the categories - the algorithm finds them)

ML = the same pipeline as DM and BI **"What we do in supervised ML is IDENTICAL to what we do in BI and data mining** - it's ALL about calculating quantities derived from patterns in existing data." Every neural network is ultimately a giant deterministic nonlinear equation: $y = F(x_0, x_1, x_2...)$

Supervised learning algorithms are "trained" using examples (DATA!) where in addition to features [inputs], the desired output [label, aka target] is known. The goal is to LEARN the patterns inherent in the training dataset, and use the knowledge to PREDICT the labels for new data.

Unsupervised learning is a type of machine learning where the system operates on unlabeled examples. In this case, the system is not told the "right answer." The algorithm tries to find a hidden structure or manifold in unlabeled data.

The goal of unsupervised learning is to explore the data to find intrinsic structures within it using methods like clustering or dimension reduction.

Semisupervised learning is used for the same applications as supervised learning. But this technique uses both labeled and unlabeled data for training - typically, a small amount of labeled data with a large amount of unlabeled data. The primary goal is unsupervised learning (clustering, for example), and labels are viewed as side information (cluster indicators in the case of clustering) to help the algorithm find the right intrinsic data structure.

With **reinforcement learning** 'RL'), the algorithm discovers for itself which actions yield the greatest rewards through trial and error. Reinforcement learning has three primary components:

1. agent - the learner or decision maker
2. environment - everything the agent interacts with
3. actions - what the agent can do

The objective is for the agent to choose actions that maximize the expected reward over a given period of time. The agent will reach the goal much quicker by following a good policy, so the goal in reinforcement learning is to learn the best policy. Reinforcement learning is often used for robotics and navigation.

Why Linear Regression Alone Won't Work

"If every neuron in every deep learning network in the world simply did linear regression and sent that out as a result - no matter how many layers, no matter how many epochs of backprop - the loss would never decrease beyond the initial loss. There would be no

learning." The nonlinear activation function is what enables learning complex patterns.

Neural Network Architecture

Input layer → hidden layers → output layer

Each layer learns a more abstract representation

Data always flows forward (input → output)

Weight corrections always flow backward (output → input) - backpropagation

"Layers and layers of neurons" - this is deep learning

What specifically makes supervised machine learning possible? Forward pass: feed input data, get prediction

Compute loss (error): how wrong was the prediction? Parabolic loss = $(\text{predicted} - \text{actual})^2 / 2$. Squared because error direction (positive or negative) doesn't matter, only magnitude

Backprop: differentiate loss with respect to each weight using chain rule. Compute gradient

Gradient descent: update each weight by small amount in direction that reduces loss

Repeat for thousands of epochs until loss curve flattens

The error curve: starts high, drops steeply, then flattens. After it flattens, more training won't help - you've reached the model's capacity. "Leave. Cool. It's as good as it'll get."

Learning rate = how big each weight step is. Momentum = how much the previous step's direction carries into the current step.

The Fourier ↔ Neural Network Analogy

Fourier synthesis: any complex periodic signal = weighted sum of sin/cos basis functions. Neural network: any complex pattern = weighted sum of nonlinear neuron outputs (activated weighted sums). Both are "big = sum of littles." The Stone-Weierstrass theorem proves: a sum of enough appropriate basis functions can approximate any continuous function. The Universal Approximation Theorem proves the same for neural networks. Neurons ARE the basis functions.

CNNs (Convolutional Neural Networks) - for Images

Architecture: First layers: classic image processing via convolution (sliding a kernel - a matrix of weights - over the image, replacing each pixel region with a weighted average of its neighbors). Used for blurring, sharpening, edge detection

Max pooling layers: Reduce image size (typically to 1/4 per step). Necessary because full-resolution images are too large for standard NN layers even on modern hardware

Final fully-connected layers: Standard NN that classifies the compressed representation

"The initial image has become a very tiny image full of pixel data. Those numbers go into the last bunch of layers called Fully Connected (FC) layers. At that point it's pure numbers, no images."

Supervised vs Unsupervised

Supervised: Training data has labels (x, y pairs). Label enables a loss function. Loss function enables backpropagation. Without labels - can't measure error - can't update weights - no learning. "The known labels help quantify the loss during each training epoch's forward pass, which in turn helps gradually reduce it via backprop."

Unsupervised: No labels. Find structure without being told what to look for. K-means clustering, LLMs (trained on next-word prediction, which is self-supervised - the "label" is just the next word in the text).

Iterative ML Algorithms (all rely on loops)

Neural Networks: the epoch loop (forward → loss → backprop → update) repeated thousands of times

K-means: the centroid reassignment loop until centroids stop moving (convergence)

Gradient descent (for linear regression on large data): the gradient step loop until minimum loss is reached

What are three DM/ML algorithms that rely on iteration? Explain where the loop is in each. A: (1) Neural networks: the epoch loop - forward pass → compute loss → backpropagation → weight update → repeat thousands of times until loss curve flattens. (2) K-means: the centroid reassignment loop - assign each point to nearest centroid → recompute centroids → reassign → repeat until centroids stop moving (convergence). (3) Gradient descent (linear regression on large data): gradient step loop - compute gradient of loss → step weights in the direction that reduces loss → repeat until minimum.

Why Visualize? Data → visualization → insight → action

Types of Visualizations: Maps for spatial data Crime, disease, business locations - all have a location, so map it. Zika virus outbreak, crime mapping in LA. "You can plan product placement by plotting where your customers are. You can figure out where to send more police by plotting where crime clusters."

Choropleth maps Quantitative variable plotted over geographic regions. Covered in detail in the spatial module. Classed vs unclassified. Prof's example: drug overdose deaths per 100K in West Virginia - the southern counties are dramatically worse than northern, visible at a glance on a map.

Spatio-temporal visualizations Data varying in both space and time. Hurricane tracks: plotted on a map showing path from formation to dissipation, color-coded by intensity (from dark red hurricane to yellow tropical storm). Can be a static map with trajectory or an animation.

Interactive visualizations The NOAA hurricane track tool - type "Katrina 2005" and see the full track, intensity changes, landfall locations. Click to explore. "100 years ago in newspapers, none of this was possible."

Treemaps, bubble charts Show proportions. Languages by speaker count as a treemap - Chinese 1.197B (largest rectangle), English 335M, Hindi 260M. Bubble charts where bubble size = quantity. Animations Census population change over decades animated - you feel where people moved, which cities grew, which shrank. Static maps can't convey this.

Six Letters: GPSETC

Governance: Weak governance once meant slow, contained breaches. Now agents act autonomously at machine speed, so governance failures cause damage faster and at greater scale before anyone notices. **Privacy:** LLMs train on scraped data, often without consent, and can leak that private/copyrighted data back through their outputs — something static databases never did, since they don't generate new content. **Security:** Security used to mean blocking unauthorized human access. Now agents can be manipulated via prompt injection into taking unauthorized actions themselves — the attack surface is the AI's reasoning, not just credentials. **Ethics:** GenAI makes deepfakes and synthetic content trivial to produce at scale, raising new "should we?" questions (consent, likeness, bias) that didn't exist before this was technically easy. **Trust:** In the agent era, trust isn't just about one company — you must trust the whole chain (LLM, every tool it calls, every org behind those tools), since one action can touch many systems silently. **Compliance:** Old laws (GDPR, HIPAA) were built for static databases, not autonomous generative systems with opaque training data — hence new frameworks like the EU AI Act.

Why GPSETC Is More Urgent Now Before LLMs: a database could leak data slowly, deliberately, one record at a time. Now: an LLM agent can generate a million convincing deepfakes in an hour, implicitly leak private training data through its outputs, take autonomous real-world actions without human review, operate at internet scale and speed.

Speed + Scale + Autonomy = every one of these six concerns is vastly more dangerous than it was in 2015.

Name and briefly explain 5 "ills" that LLMs can cause (for a GPSETC/AI ethics question). A: (1) Deepfakes / impersonating public figures - generate convincing fake video/audio of real people for propaganda. (2) Crafting convincing scam/phishing emails - personalized, grammatically perfect, hard to detect. (3) Generating fake medical advice - plausible-sounding but dangerous health information. (4) Automated social media manipulation - generate thousands of fake accounts and posts to shift public opinion. (5) Generating biased or prejudiced content - training data biases get amplified at scale. Also valid: fake research papers, fake product reviews, fake legal documents, fake evidence or alibis, generating scripts for scam calls.

The History of GenAI in One Line

NN that classifies → GANs made them generate → transformers scaled everything → RAG added external knowledge → agents gave them tools and autonomous loops.

Generator (the student): Takes random noise as input, tries to produce fake data (faces, images)

Discriminator (the teacher): Trained on real data, tries to distinguish real from fake. Gives the generator a loss signal

They train together in a competitive loop. Generator improves at fooling discriminator. Discriminator improves at detecting fakes. Eventually the generator produces photorealistic faces from pure random noise.

Site: thispersondoesnotexist.com - every refresh generates a new face from nothing but noise. "It did not steal a photo and Photoshop it. The entire face was created in the background." Two separate loss functions - one trains the generator, one trains the discriminator. "The G in GAN is the same G as in GenAI." Before GANs, neural networks could only classify, not generate. GANs changed that.

Describe the GAN training loop. A: Generator (student) takes random noise as input, tries to produce fake data (faces, images). Discriminator (teacher) trained on real data, tries to distinguish real from fake - gives the generator a loss signal. They train together competitively: Generator improves at fooling the Discriminator; Discriminator improves at detecting fakes. Eventually the Generator produces photorealistic output from pure noise. Two separate loss functions - one trains each. Before GANs, NNs could only classify; GANs enabled generation.

Transformers & LLMs

Encoder-decoder architecture. Training = predict the next token given all previous tokens. This is self-supervised - no external labels needed; the "label" is just the next word in the text. After pre-training, you only need the decoder for inference (that's why deployed models like GPT are "decoder-only"). Variational autoencoders make the latent space continuous so you can generate new tokens by interpolating between points.

Fine-Tuning & LoRA

After pre-training, fine-tune on domain-specific data to make the model better at one task. LoRA (Low-Rank Adaptation, Microsoft): instead of retraining all weights, only modify the top layers. Produces a small adapter file you download on top of the base model. "Download this file, it'll answer C++ questions better."

RAG (Retrieval-Augmented Generation)

LLMs have a knowledge cutoff and don't know your company's internal documents. RAG: build a vector database (or knowledge graph) from your documents; at query time, retrieve relevant chunks and feed them as context to the LLM. The LLM answers based on your retrieved data.

Vector RAG: embed documents as vectors, do similarity search

Graph RAG: build a knowledge graph from documents, traverse it for answers (GraphRAG by Microsoft)

Hybrid RAG: combine both

RAG is more efficient than fine-tuning for domain-specific/frequently-changing knowledge

"RAG basically means almost whatever you want to do - except touch the actual LLM."

Agents and the ReAct (reason, act, observe, repeat)

An agent = an LLM wrapped in an infinite loop:

while task_not_complete:

 reason about current state (the "think" step)
 decide which tool to call (the "plan" step)
 call tool (the "act" step - database, API, code, email...)
 observe result
 update state

This is called the ReAct loop (Reason + Act). Also called the "Ralph's loop". "Everything is a Ralph's loop" - every software development pipeline, every autonomous agent pattern.

Key innovation: the agent doesn't come back to you until the task is done. "Give it a one-shot thing and just go away. No need to babysit." A separate critic component checks the output before the agent declares success.

MCP (Model Context Protocol)

"Like a USB port for AI agents." A standardized interface that lets any agent talk to any service in plain text. Instead of writing custom API code per service, you write an MCP server for your service. The agent queries it in natural language; MCP translates to actual API calls.

The **full modern stack (bottom to top): Database → REST API → GraphQL → MCP → Agent**

Evolution of prompting: "In the beginning, you just chatted with the LLM. Then people added context manually (prompt engineering). Then RAG automated context. Then agents added tools and loops. Then MCP standardized how agents talk to tools."

AI + Databases (DATA → DAgtA)

How does DATA become DAgtA? What are three ways agentic AI and databases collide?A: (1) Text-to-SQL via MCP - describe query in English, agent generates and executes SQL. (2) Agents build knowledge graphs from unstructured text (Neo4j + agents). (3) AI-native databases - TiDB (SQL + AI), FalkorDB (graph + AI) - can be created and managed by agents on demand. Also: agents automatically generate ER schemas, agents optimize slow queries and suggest indexes, RAG uses vector DB as agent memory.

How agentic AI and databases collide:

Agents create ER schemas and data models automatically

Agents build knowledge graphs from unstructured text (Neo4j + agents)

Agents do text-to-SQL via MCP (ask in English, SQL runs for you)

Agents optimize slow queries and suggest indexes

AI-native databases: TiDB (SQL + AI), FalkorDB (graph + AI) - can be created and managed by agents on-demand

GraphQL: A wrapper on top of REST. Problem with REST: you ask for one thing and get back 100 fields, most of which you don't need. GraphQL: ask for exactly the fields you want. The GraphQL layer calls multiple REST endpoints behind the scenes, filters, returns only what you asked for. "Like sending a partial JSON where you only specify the keys and ask the server to fill in the values." Backward compatible with REST - someone who knows REST can still use it.

MCP: A wrapper on top of GraphQL/REST. Ask in plain natural language; MCP translates to API calls. The newest layer. "This is what makes agents actually useful for everyday people."

Architecture progression: ODBC → JDBC → REST → GraphQL → MCP (each layer wraps and extends the previous, nothing is thrown away).

Four-tier architecture: Browser/Client → Web Server → Web Application Server → Database. Each tier handles part of the work so no single tier is overloaded.

Distributed Database?Data physically stored on multiple machines (nodes) connected by a network, but appearing logically as a single database to users. Users don't know (and don't need to know) where the data actually lives.

Why DDBMS? Rapid ad-hoc access to data/internet speed decision making; distributed data access was needed to serve dispersed business units(globalization); mobile wireless revolution; usage of application as a service; focus on mobile BI

TP: Transaction Processor - receives data requests (from an application), and requests data(from a DP); also known as AP (Application Processor) or TM (Transaction Manager). A TP is what fulfills data requests on behalf of a transaction.

DP: Data Processor - receives data requests from a TP (in general, multiple TPs can request data from a single DP), retrieves and returns the requested data; also known as DM (Data Manager). Prof's example: Google indexes the entire web. One machine can't hold it. They fragment it across thousands of machines worldwide. Your search hits all of them simultaneously. That's a distributed database in action.

Distribution transparency levels:

Fragmentation transparency: end user does not know that the data is fragmented.

Location transparency: end user does not know where fragments are located.

Location mapping transparency: end user does not know how fragments are mapped.

Distrib. transparency is supported via a distributed data dictionary (DDD) [aka DDC], which contains the distributed global schema which local TPs use to translate user requests for processing by DPs.

Distributed db design: data fragmentation(divide), replication(copy), allocation(when to store copies)

Data Fragmentation (Partitioning): Splitting one large table across multiple nodes so each node only holds part of the data.

Strategies: Horizontal fragmentation: Split by rows. Fragment 1 = rows 1–100K, Fragment 2 = rows 100K–200K, etc. All fragments have the same columns. This is what enables parallel processing (MapReduce). Prof's example: split a 100K-employee salary table into 3 fragments across 3 machines → calculate average salary 3× faster.

Vertical fragmentation: Split by columns. Fragment 1 = the columns you access all the time. Fragment 2 = the rarely-used archival columns. Still the same rows, but split by what's accessed. Prof's example: USC could put current grad student transcript columns in one fast fragment, and rarely-needed undergrad transcript columns in another, slower storage fragment.

Mixed fragmentation: First apply vertical, then horizontally fragment the important columns. Most practical strategy. The key insight: to the application, it's still logically one table. The distribution is transparent.

Data Replication: Keeping multiple copies of the same data on different nodes, at different geographic locations.

Full replication: Every node has a complete copy. Best for availability and read performance, but expensive to keep in sync

No replication: One copy only. Cheapest, but catastrophic if that node fails

Partial replication (Goldilocks): Replicate the most important/frequently accessed data. Lose less critical data less often. What most real systems do.

Push vs Pull replication: Push = when data changes on master, immediately send copies to replicas. Pull = replicas periodically ask master for updates.

Master-Slave replication: One master accepts writes, many slaves receive copies. Most common pattern.

CAP Theorem: All distributed databases must trade off between three properties - and you can only fully guarantee two at a time:

Consistency: No matter which node you connect to, you always see the same, up-to-date data. One source of truth.

Availability: The system always responds to requests. No node is ever "down for maintenance."

Partition Tolerance: The system continues operating even if network links between nodes fail (some nodes become isolated).

The trade-off: In the real world, network partitions WILL happen. So P is not really a choice - you always need it. The real choice is: when a partition occurs, do you sacrifice C (availability-focused) or A (consistency-focused)?

Traditional RDBMS: choose CA. Strong consistency + high availability, but assumes no network partitions (fine for a single-machine DB)

NoSQL DBs: choose AP. Available even during partitions, but data may be temporarily stale. Hence BASE instead of ACID

Prof's fintech example: stock trading systems need consistency above all. When exchange rates change, every trading server must see the same value instantly, or one trader gains while another loses. They'll accept brief downtime for consistency.

Prof's e-commerce example (Amazon): for a billion users browsing products, availability wins. If a user temporarily sees yesterday's price, that's OK. It will become consistent eventually. No availability = lost revenue.

The core JSON rules:

Everything lives inside outer { } curly braces

Every key must be a string in double quotes: "key": value

Values can be: string "hello", number 42, boolean true, array [...], or another object {...}

Arrays use [] square brackets, elements separated by commas

Key-value pairs separated by commas; the last pair has no trailing comma

Objects and arrays can be infinitely nested inside each other - this is the true power of JSON

How to convert a table to JSON: Each row becomes a { } object. Column names become keys (strings). Cell values become values. All rows go into an array []. The whole thing is wrapped in an outer object with a key naming the table:

```
{
  "employees": [
    { "id": 1, "name": "Alice", "dept": "Engineering" },
    { "id": 2, "name": "Bob" }
  ]
}
```

Note: Bob has no dept - in JSON you simply omit the key entirely (no null needed). This is schema-on-read.

How to convert a SQL query to JSON: The three parts of a SELECT statement become three keys:

```
{
  "SELECT": ["AccountID", "AccountBalance"],
  "FROM": "Customers",
  "WHERE": [{"Country": "US" }]
```

```
}
}
How to convert a function to JSON: A function has 3 pieces:
return type, parameter list, body:
{
  "methodName": "add2Nums",
  "outputType": "int",
  "inputParams": [
    { "name": "a", "datatype": "int" },
    { "name": "b", "datatype": "int" }
  ],
  "body": "return (a+b)"
}
```

JSON vs GraphQL: Similarity: Both use curly-brace key-value syntax. Both are hierarchical. Both used for client-server data exchange over the web. Both human-readable. Difference: JSON carries both keys AND values (it's a data format). GraphQL carries only keys - you specify what you want, the server fills in the values, usually returning JSON. GraphQL is a query language, not a data format. GraphQL is more efficient - you get exactly what you ask for, no over-fetching.

MQTT: lightweight pub-sub protocol for IoT (low-bandwidth/unstable networks). **SOAP:** XML-based web services protocol (WSDL→Request→Response). **Webhook:** event-triggered push notification - server sends data to a client URL automatically when something happens (vs. client polling). **WebSocket:** persistent, two-way, real-time connection between client and server over a single long-lived connection (handshake → persistent connection → closure).

Relational Table vs Vector DB

Given 5 numerical features A B C D E - how do you search in a 10M-row relational table vs a 5D vector space with 10M points?

Similarities:

Both represent records with multiple features/attributes/dimensions. A row in a relational table with 5 columns = a point in 5D space. Same data, different representation.

Both need indexes to speed up search (B-tree/hash for relational; HNSW for vector)

Both use hierarchical indexing structures internally (R-trees ↔ HNSW)

Searching a relational table: Use SQL with exact conditions or ranges: WHERE salary > 50000 AND dept = 'Engineering'. Logical, exact, condition-based. Finds exact matches or ranges.

Searching a vector DB: Use similarity search - find the k nearest neighbors to a query vector. Distance metric: Euclidean distance or cosine similarity. Results are approximate (ANN - Approximate Nearest Neighbor). Index: HNSW (Hierarchical Navigable Small World).

difference: Relational DB → exact / logical search (WHERE conditions, equality/range). Vector DB → similarity search (proximity in high-dimensional space, approximate). One is deterministic, the other is probabilistic. Relational = "give me rows WHERE X = Y". Vector = "give me the 5 most similar things to this query."

Underfitting (high bias, low variance): The model is too simple to capture the real pattern. It fails to learn the relationship between features and the target. Poor accuracy on BOTH training data and test data. Example: fitting a straight line to data that follows a parabolic curve - the line is biased towards simple, low values and misses the real trend entirely. The rubric's killer specific point: a linear fit to goal-scoring-over-time data would predict negative goal counts before ~25 minutes - physically impossible, clearly wrong.

Overfitting (low bias, high variance): The model is too complex - it memorizes the training data, including noise and outliers. Excellent accuracy on training data, poor accuracy on unseen test data. The model has no generalizability. Like memorizing every exam answer without understanding the subject.

Left diagram (bad): Linear fit / underfit - straight line through data that clearly isn't linear

Right diagram (better): Non-linear fit (polynomial, Poisson, quadratic etc.) - curved line that captures the true pattern

Why the non-linear fit is preferable: It captures the true underlying relationship in the data. It will generalize to new/unseen data because it learned the actual pattern, not noise and not an oversimplification.

The bias-variance tradeoff:

High bias (underfit) → model too simple → misses the pattern

High variance (overfit) → model too complex → memorizes noise

Goal: find the middle ground - fits the general trend without memorizing every data point

Linear regression example he uses: High school GPA predicts university GPA. Fit $y = mx + c$. New student comes in, plug in their high school GPA, get predicted university GPA. That model (the line equation) replaces the raw data - data → model.

Non-linear regression example: TV viewing hours as function of age - U-shaped parabola (high as child, low as working adult, high again as retiree). Fit $y = ax^2 + bx + c$. Can also fit non-linear surfaces in multiple dimensions (e.g., predicting salary from years of experience AND education level).

Data Visualization: Then vs Now:What was possible in 1930s newspaper charts (flat, static, print):Static image only; No interactivity; No animation; No real-time updates; No color (often); No clicking, drilling, filtering

Two things we can do TODAY that weren't possible in newsprint (staying flat/2D): Interactive visualization - drill down, roll up, filter, click on data points for detail. Users get a personalized

view. Example: NOAA hurricane track tool - type "Katrina 2005" and see the full track interactively.

Animated visualization - show change over time as animation. Census population shifts animated over decades. Hurricane approaching landfall animated in real time.

Real-time visualization - data updates dynamically as new data arrives. Stock tickers, live COVID dashboards, live crime maps.

Three ways to TRANSCEND flat display entirely:

3D printing - physically print the data

Haptics - feel the data through vibration/force feedback

Augmented Reality (AR) - overlay data on the real world

Virtual Reality (VR) - immerse yourself in the data

Holography - 3D projection without a screen

Projection mapping - project data onto physical surfaces (buildings, domes)

Dome display- data wrapped around the inside of a dome.

Examples for the inside of a dome: sunburst charts, zoomable circle packing, any visualization with circular symmetry. For the outside: weather patterns, animal migration, artistic/promotional content.

Intelligent comprehension / Q&A - ask questions about data in natural language and get answers (agentic AI over data)

Why is it called "unsupervised"? Because there is no labeled data and no external "supervisor" (teacher) telling the algorithm what the right answer is. The algorithm finds patterns on its own, without pre-defined categories. No labels → no loss function based on right/wrong → the algorithm clusters, groups, or finds structure purely from the input data itself.

The ONE canonical unsupervised DM algorithm (pre-LLM era):

K-means clustering. For decades, this was essentially the only widely taught unsupervised learning algorithm. You give it data and a number k, it finds k clusters, no labels needed.

What changed recently - why did unsupervised learning suddenly matter hugely? LLMs (Large Language Models) / Transformers. You cannot label all of human language - there are trillions of words and you can't tag every noun, verb, relationship.

So LLMs are trained on raw unlabeled text using self-supervised (a form of unsupervised) learning: predict the next token. The "label" is just the next word in the text - no human annotation needed. This made LLMs possible and brought unsupervised learning to the center of AI. **The data connection:** LLMs need MASSIVE amounts of unlabeled textual data - the entire internet, books, code, etc. The transformer architecture learns language structure (syntax, semantics, relationships) from this raw data without anyone manually labeling anything. More data → better model. This is the defining characteristic of the LLM era.

How supervised ML helps with DB performance tuning:

Traditional performance tuning relies on collected query statistics

and hand-crafted rules from expert DBAs. Supervised ML can automate this:

Cardinality estimation: ML learns from past queries how many rows a query will actually return - better estimates → optimizer picks better query plans

Index recommendation: ML analyzes historical query patterns, identifies which columns are most frequently used in WHERE clauses, recommends creating indexes on them - reduces full table scans

Query plan selection: ML learns from past execution times of different plans, predicts which plan will be fastest for a new query without actually running all alternatives

Workload analysis: ML identifies patterns in when queries run, which tables are hot, enabling proactive resource allocation

The supervised part: training data = past (query, execution time, plan chosen) triples. Label = whether the plan was good or bad. The model learns to predict good plans.

How GenAI helps with DB performance tuning?Natural

language to SQL: Users describe what they want in plain English; GenAI generates optimized SQL - removes the need for users to write performant queries manually. Synthetic data generation: GenAI generates realistic test data for stress-testing query performance without using real customer data. Explainable performance reports: GenAI translates dry statistics ("index scan took 3.2s with 45M rows examined") into plain-English explanations and actionable recommendations for DBAs. Automated documentation: GenAI generates schema documentation, query comments, and data dictionaries automatically

What is data science? In what sense has it always existed? In what sense is it new? A: Data science = interdisciplinary field

combining math, statistics, CS, and domain knowledge to extract meaningful information from large, sophisticated datasets. Always existed: statistics is the mature older discipline - methods for collecting and analyzing data to make decisions have existed for decades/centuries. New: massive computing power, massive digital data volume (the web created it), new tools (GPU, cloud, algorithms, interactive viz). The methods are old; the scale, infrastructure, and digital data explosion are new.

Describe the evolution of DB connectivity from pre-WWW to now. A: Pre-WWW: Microsoft's ODBC (PC-only, not truly "open"), DAO, JET, RDO - and Java's JDBC (cross-platform, runs on any OS). On a PC, everything ultimately goes through ODBC.

Post-WWW: REST APIs (stateless HTTP requests, returns full JSON regardless of what you need), then GraphQL (wrapper on REST - ask for exactly the fields you want, server calls multiple REST endpoints behind the scenes), then MCP (natural language wrapper on GraphQL/REST - agents query in English, MCP translates to API calls). Architecture progression: ODBC → JDBC

→ REST → GraphQL → MCP. Nothing is thrown away; each layer wraps the previous.

What makes microservices architecture flexible?A: Each service is small, independent, and loosely coupled - it can use its own tech stack, programming language, and database without affecting other services. One service can be updated, redeployed, or scaled independently. No shared dependencies means a change in one service doesn't break others. They communicate via an API gateway.

How do microservices actually work?A: Services run in containers (Docker); a container orchestrator (Kubernetes) manages them - spins up more containers when demand is high, removes them when demand drops (elastic cloud computing). Services are deployed on cloud (AWS, GCP, Azure). A client calls the API gateway; the gateway routes the request to the appropriate microservice container. MCP is the newest layer that lets agents call microservices in natural language.

"Many problems in CS can be solved by introducing a level of indirection." Apply this to: (a) dataflow graphs on Hadoop, (b) Kafka pub-sub, (c) agents with MCP. A: (a) Dataflow (Pig/Hive on Hadoop): the DAG compiler/optimizer sits between the frontend high-level graph and the parallel backend. This decouples the developer from the execution engine - swap backends (Hadoop → Spark) without rewriting the dataflow logic. (b) Kafka pub-sub: the broker/topic sits between producers and consumers. They never talk directly - decoupled in time (consumer doesn't need to be online), space (they don't know each other), and sync. (c) MCP/agents: MCP sits between the LLM and real systems (databases, APIs, tools). The LLM never calls systems directly - swap tools or change APIs without retraining the model.

H3 (Hexagonal Hierarchical Spatial Index) Divides the globe into a hierarchy of hexagonal cells instead of square/rectangular grid cells.**Why hexagons over squares/rectangles?** In a square grid, neighboring cells aren't all equidistant - edge-adjacent neighbors (4 of them) are closer than corner-adjacent neighbors (4 more), making "nearest neighbor" calculations inconsistent. In a hexagonal grid, all 6 neighbors of a cell are equidistant from its center - uniform adjacency makes distance-based queries (ride-hailing surge pricing zones, delivery radius, dispatch) cleaner and more accurate.Like a quadtree, H3 is hierarchical - cells can be subdivided into smaller hexagons for finer resolution where needed. A traditional soccer ball = a truncated icosahedron: 20 regular hexagon faces + 12 regular pentagon faces (32 faces total, 90 edges, 60 vertices). Each pentagon is surrounded entirely by hexagons. This is one of the 13 Archimedean solids.

Democracy vs. Dictatorship in Distributed Consensus: 2PC (single coordinator) = "dictatorship" model: One coordinator unilaterally decides commit/abort for everyone after collecting votes.

Fast and simple, but if the coordinator crashes mid-protocol, participants can be left blocked indefinitely waiting for a decision (single point of failure).

Consensus protocols (Raft, Paxos) = "democracy" model: No single coordinator holds permanent authority. Nodes propose and vote; a majority quorum (not unanimity) decides the outcome. Tolerant of individual node failures - if one node (even a "leader") crashes, the remaining majority can still make progress and elect a new leader. Key contrast: 2PC requires unanimous agreement (any single "no" vote aborts everything) and depends on one coordinator's survival. Raft/Paxos only need a majority and survive minority node failures - directly trading the centralized/fast dictatorship model for the decentralized/fault-tolerant democracy model.

LSM (Log-Structured Merge) Trees + SSTables (Sorted String Tables)-a pair of data structures optimized for write-heavy workloads, used as an alternative to B-tree indexing.

The core idea:B-trees are great when you read a lot and modify data occasionally; they keep everything sorted in place, which makes in-place updates expensive (you have to find the exact spot and insert/rebalance). LSM trees flip this: instead of writing in-place, new writes are appended sequentially (fast, no seeking/rebalancing), and the data only gets sorted and merged into SSTables (Sorted String Tables) later, in the background.

Why this matters: "Suppose you write data a lot and you modify it once in a while, rather than have indexing be done by B-trees, use this pair of data structures to more easily access data that you hardly modify." i.e., LSM/SSTables trade a bit of read efficiency for much faster, cheaper writes, since appends are sequential (disk-friendly) instead of random-access updates.

Real-world example: Cassandra uses SSTables to persist data, implemented via LSM trees. **Key-value DBs** rely on LSM trees for rapid key querying on disk; this is why k-v stores can offer fast writes/reads despite having no schema and no B-tree-style structure.

ETL (Extract, Transform, Load) = "schema-on-write" You design the schema (table structure) FIRST;decide all your columns ahead of time — THEN load data into that pre-built structure. This is the classic, traditional data warehouse approach: very RDBMS-like, historical record of transactions.

Extract: pull data from operational sources (only extract what the schema needs)

Transform: convert all data into one common format before merging e.g., date formats (US "4/2" vs. European "4/2" mean different dates), currencies (RMB vs. USD vs. EUR), units

Load: insert the cleaned, transformed data into the warehouse table

End warehouse properties: integrated, subject-oriented, time-variant, nonvolatile

ELT (Extract, Load, Transform) = "schema-on-read" = "reverse ETL"You flip the order: dump raw, unstructured data into storage FIRST (no schema decided yet) load it as-is and only AFTER it's stored do you look at a sample of what's there, figure out what columns/schema make sense, and transform it into a structured form. This is the modern **Data Lake approach**: dump everything raw (JSON, CSV, PDFs, tweets, Slack messages, WhatsApp exports anything) into one unstructured "lake," then pull out and structure only what you need for a given analysis (BI, ML, etc.)

Schema-on-read is not truly "schema-less" the schema is implicit, living in the application code that reads/writes the data, not in the database itself. The DB "doesn't care" what it's storing; meaning is imposed by the code.

How is "schema-on-read" conceptually similar to how a NoSQL document store (e.g., MongoDB) handles data? A: Both delay structure decisions: NoSQL document stores don't enforce a fixed schema across records (one document can have fields another lacks), and schema-on-read similarly doesn't define structure until the data is actually queried/read — in both cases, the application code, not the database engine, is what imparts meaning/structure to the raw data

GAN vs Diffusion vs VAE: GAN consists of a Generator that takes random noise and tries to produce fake data, and a Discriminator that is trained on real data and tries to distinguish real from fake, giving the Generator a loss signal. The two networks train together adversarially using two separate loss functions, with the Generator getting better at fooling the Discriminator and the Discriminator getting better at detecting fakes, until the Generator can produce photorealistic output from pure noise.

Diffusion uses a single network instead of two competing ones. Training data is created by taking real images and progressively adding noise until they become pure noise, producing pairs of (noisier image, less-noisy image). The network learns to reverse this process, so at inference time it can start from pure noise plus a text prompt and iteratively denoise it step by step until a coherent image matching the prompt emerges. This is the underlying mechanism of Stable Diffusion, and the same iterative denoising approach can also be used for text generation.

A Variational Autoencoder (VAE) is an encoder-decoder pair where the encoder maps an input to a distribution in latent space rather than a single point, which makes the latent space continuous and allows interpolation between points. For example, encoding blues, jazz, and Indian classical music as three points in latent space means a new point near all three can be decoded into music that blends all three styles. The word "variational" refers to this distribution-based, interpolatable latent space, and "auto" refers to the encoder and decoder being trained together in one architecture.

What do eventual consistency (CAP theorem) and closure (spatial algebra) have in common?A: Both describe a system temporarily leaving a "clean" guaranteed state and needing extra work to return to it. Eventual consistency means replicas may briefly disagree after a write, syncing up later. Closure breaking means an operation like distance returns a number instead of a geometry, stepping outside the closed algebra until it's reinterpreted back into spatial form. Both trade a strict guarantee for flexibility/speed, accepting temporary inconsistency that's resolved afterward rather than instantly.

Q: How does "introducing a level of indirection" apply to (A) front-end dataflow graphs on backend parallel platforms, (B) pub-sub systems, and (C) agents/tool-calling?A. Dataflow graphs (Pig, GraphLINQ) on backend platforms (Hadoop, Naiad):Muskeeter-A compiler/optimizer sits between the dataflow graph and the execution engine. This decouples the developer from the backend, so you can swap engines without rewriting the dataflow logic. B. Pub-sub systems:kafka-A broker/topic sits between publisher and subscriber. They never talk directly. This decouples them in time, space, and synchronization. C. Agents and tool-calling (MCP):MCP sits between the LLM and real systems (APIs, databases, tools). The LLM never calls them directly, it just expresses intent in language. This lets tools/APIs change without retraining the LLM.

Why can hallucination never be fully eliminated from an LLM, architecturally?A: Hallucination is structural, not a fixable bug. When a prompt lands far from dense training data, the model's similarity/distance calculation still finds the nearest points and interpolates an answer — it has no built-in mechanism to recognize "this is outside my knowledge" the way a human would. It always answers confidently, regardless of how well-represented the prompt was in training data.

What is a "world model," and why do LLMs trained only on text need one for physical tasks?A: Text alone gives no physical common sense — a text-only model doesn't "know" that a dropped object falls and comes to rest. A world model is trained, often on video, to learn how the physical world behaves so it can predict outcomes of physical actions. Embodied AI takes a world model and puts it on a robot, so it perceives and acts from a first-person point of view instead of just answering prompts.

What allows an agent to resume a task across multiple loop iterations rather than starting over?A: The accumulated state across loop iterations is saved in what's called a "harness." This lets the agent resume an interrupted task instead of restarting from scratch, and the saved state can be reused on future tasks to speed up the workflow.